

Lab 1

name: WangShiyu nuist_id: 202383910020 setu_id: W20109917

Experiment Purpose

1. Docker and Containerization

Understand the fundamental concepts of Docker containerization technology, learn to install and configure Docker Desktop on Windows, and master the use of Docker Compose for deploying containerized applications.

2. Gitea Deployment

Learn to deploy the Gitea using Docker, understand container configuration including persistent storage and port mapping, and complete the initial setup of the Gitea server.

3. Git Large File Storage (LFS)

Understand the limitations of traditional Git in handling large binary files and the working principles of Git LFS. Master the installation and configuration of Git LFS on both client and server sides.

4. Practical Application of Git LFS

Learn to configure LFS tracking rules, manage large files using Git LFS workflow, and verify successful integration of Git LFS with Gitea for effective version control of large files.

Experiment Process

1. Docker Desktop Installation and Configuration

Since the experiment was conducted on a Windows operating system, the first step was to set up the proper environment for running Docker containers.

(1) WSL2 Installation

Windows Subsystem for Linux (WSL2) was installed to provide a Linux-compatible environment for Docker. The installation was performed by executing the following command in Windows PowerShell with administrator privileges.

```
PS C:\Windows\system32> wsl --install
正在下载: 适用于 Linux 的 Windows 子系统 2.6.3
正在安装: 适用于 Linux 的 Windows 子系统 2.6.3
已安装 适用于 Linux 的 Windows 子系统 2.6.3。
正在安装 Windows 可选组件: VirtualMachinePlatform

部署映像服务和管理工具
版本: 10.0.26100.5074

映像版本: 10.0.26200.8037

启用一个或多个功能
[=====100.0%=====]
操作成功完成。
请求的操作成功。直到重新启动系统前更改将不会生效。
请求的操作成功。直到重新启动系统前更改将不会生效。
```

After the installation completed, the system was restarted to finalize the WSL2 setup.

```
PS C:\Windows\system32> wsl --version
WSL 版本: 2.6.3.0
内核版本: 6.6.87.2-1
WSLg 版本: 1.0.71
MSRDC 版本: 1.2.6353
Direct3D 版本: 1.611.1-81528511
DXCore 版本: 10.0.26100.1-240331-1435.ge-release
Windows: 10.0.26200.8246
```

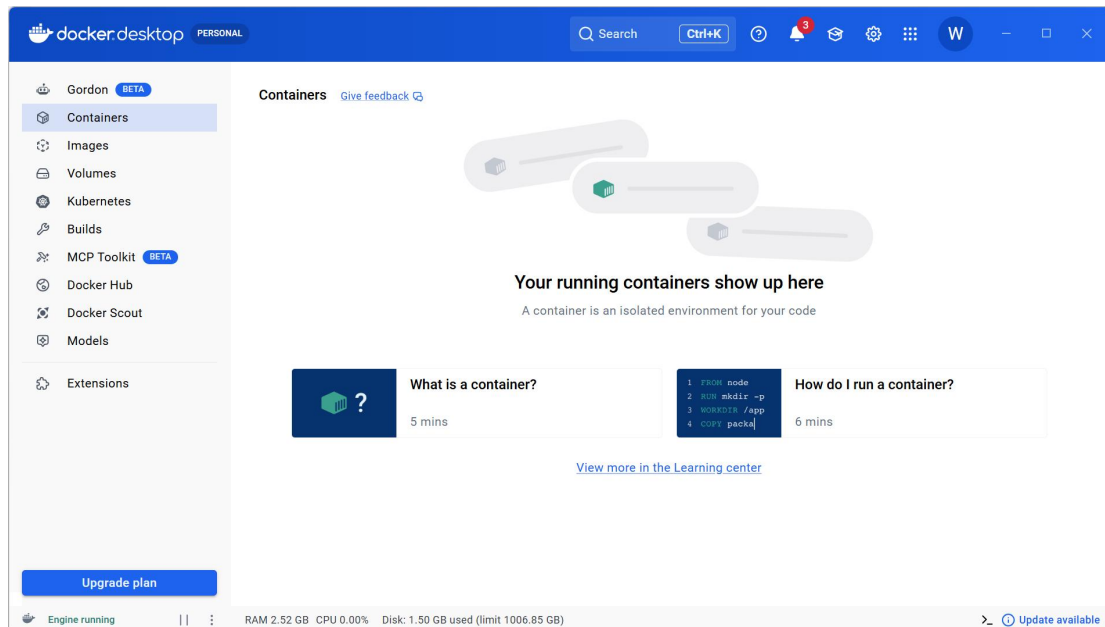
(2) Docker Desktop Installation

Docker Desktop for Windows was downloaded from the official Docker website. During the

installation process, the option "Use WSL 2 instead of Hyper-V" was selected to ensure compatibility with the WSL2 backend. After installation, Docker Desktop was launched and the Docker service was verified to be running properly by checking the Docker Dashboard.

(3) Docker Hub Account Registration

A Docker Hub account was registered as required by the installation instructions. The Docker Desktop application was then logged in using the created credentials.



2. Gitea Installation Using Docker

(1) Project Directory Setup

A working directory was created to organize all Gitea-related files.

```
PS C:\Users\WangShiyu> mkdir gitea-lab

  目录: C:\Users\WangShiyu

Mode                LastWriteTime         Length Name
----                -
d-----          2026/4/26      15:06         gitea-lab

PS C:\Users\WangShiyu> cd gitea-lab
PS C:\Users\WangShiyu\gitea-lab> mkdir -p gitea

  目录: C:\Users\WangShiyu\gitea-lab

Mode                LastWriteTime         Length Name
----                -
d-----          2026/4/26      15:06         gitea
```

(2) Docker Compose Configuration

A docker-compose.yml file was created to define the Gitea container configuration.

```
PS C:\Users\WangShiyu\gitea-lab> notepad docker-compose.yml
```

```
文件 编辑 查看

networks:
  gitea:
    external: false

services:
  server:
    image: docker.gitea.com/gitea:1.26.1
    container name: gitea
    environment:
      - USER_UID=1000
      - USER_GID=1000
    restart: always
    networks:
      - gitea
    volumes:
      - ./gitea/data
      - /etc/timezone:/etc/timezone:ro
      - /etc/localtime:/etc/localtime:ro
    ports:
      - "3000:3000"
      - "222:22"
```

The configuration specified Gitea version 1.26.1, set up port mapping for HTTP access (port 3000) and SSH access (port 222), and enabled Git LFS support through the environment variable GITEA__server__LFS_START_SERVER=true.

(3) Container Deployment

The Gitea container was launched using Docker Compose.

```
PS C:\Users\WangShiyu\gitea-lab> docker compose up -d
[+] up 9/9
✔Image docker.gitea.com/gitea:1.26.1 Pulled 11.3s
✔Network gitea-lab_gitea Created 0.0s
✔Container gitea Started 0.7s
```

The container status was verified.

```
PS C:\Users\WangShiyu\gitea-lab> docker compose ps
NAME IMAGE COMMAND SERVICE CREATED STATUS PORTS
gitea docker.gitea.com/gitea:1.26.1 "/usr/bin/entrypoint..." server 2 minutes ago Up About a minute 0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp, 0.0.0.0:222->22/tcp, [::]:222->22/tcp
```

(4) Gitea Initial Configuration

A web browser was used to access http://localhost:3000 to complete the Gitea installation wizard.

Database Type	SQLite3 (default)
Server Domain	localhost
HTTP Port	3000
Base URL	http://localhost:3000/
Administrator account credentials were created	

初始配置

如果您正在使用 Docker 容器运行 Gitea，请务必仔细阅读 官方文档 后再对本页面进行填写。

数据库设置

Gitea 需要使用 MySQL、PostgreSQL、Microsoft SQL Server 或 TiDB (MySQL 协议) 等数据库

数据库类型 * SQLite3

数据库文件路径 * /data/gitea/gitea.db

SQLite3 数据库的文件路径。如果以服务的方式运行 Gitea，请输入绝对路径。

一般设置

站点名称 * Gitea: Git with a cup of tea

您可以在本输入您公司的名称。

仓库根目录 * /data/git/repositories

所有远程 Git 仓库将保存到此目录。

LFS根目录 * /data/git/lfs

存储为Git LFS的文件将被存储在此目录。需空禁用LFS

以用户名运行 * git

输入 Gitea 运行的操作系统用户名。请注意，此用户必须具有对仓库路径的访问权限。

服务器域名 * localhost

服务器域的域名或主机地址。

SSH 服务器端口 * 22

SSH 服务器端的端口号，为空则禁用它。

HTTP 服务器端口 * 3000

Gitea Web 服务器端将侦听的端口号。

输入 Gitea 运行的操作系统用户名。请注意，此用户必须具有对仓库路径的访问权限。

服务器域名 * localhost

服务器域的域名或主机地址。

SSH 服务器端口 * 22

SSH 服务器端的端口号，为空则禁用它。

HTTP 服务器端口 * 3000

Gitea Web 服务器端将侦听的端口号。

基础URL * http://localhost:3000/

用于 HTTP (S) 代理和邮件通知的基本地址。

日志路径 * /data/gitea/log

日志文件将写入此目录。

☐ 启用更新检查

通过连接到 gitea.io 定期检查新版本发布。

可选设置

☒ 电子邮件检查

☒ 服务器和第三方服务设置

☒ 管理员帐号设置

这些配置选项将写入以下位置: /data/gitea/conf/app.ini

立即安装

After clicking "Install Gitea", the installation was completed and the Gitea dashboard became accessible.



3. Git LFS Configuration and Verification

(1) Git LFS Installation

Git LFS (Large File Storage) was installed on the Windows system by downloading the installer from the official Git LFS website.

```
PS C:\Users\WangShiyu> git lfs version
git-lfs/3.7.1 (GitHub; windows amd64; go 1.25.1; git b84b3384)
```

(2) LFS Server-Side Configuration

Git LFS support was enabled on the Gitea server through the environment variable configured in the docker-compose.yml file. This allowed Gitea to properly handle large file storage and management.

```
PS C:\Users\WangShiyu\gitea-lab> notepad gitea\gitea\conf\app.ini
PS C:\Users\WangShiyu\gitea-lab> docker compose restart
[+] restart 0/1
- Container gitea Restarting
```

(3) Repository Creation

A new repository named "test-lfs" was created through the Gitea web interface for testing Git LFS functionality.



(4) Repository Cloning and LFS Setup

The repository was cloned to the local machine.

```
PS C:\Users\WangShiyu> git clone http://localhost:3000/WangShiyu/test-lfs.git
Cloning into 'test-lfs'...
warning: You appear to have cloned an empty repository.
PS C:\Users\WangShiyu> cd test-lfs
```

(5) LFS Tracking Configuration

Large binary files with the .bin extension were configured to be tracked by Git LFS.

```
PS C:\Users\WangShiyu\test-lfs> git lfs track "*.bin"
Tracking "*.bin"
```

The tracking configuration was verified.

```
PS C:\Users\WangShiyu\test-lfs> git lfs track
Listing tracked patterns
*.bin (.gitattributes)
Listing excluded patterns
```

4. Large File Creation and Upload Test

(1) Creating a 1GB Test File

A 1GB binary file was created using the Windows fsutil command. This command created a file named large_file.bin with a size of exactly 1,073,741,824 bytes (1GB).

```
PS C:\Users\WangShiyu\test-lfs> fsutil file createnew large_file.bin 1073741824
已创建文件 C:\Users\WangShiyu\test-lfs\large_file.bin
```

(2) Committing and Pushing the Large File

During the push operation, Git LFS automatically detected that the .bin file matched the tracking pattern and uploaded the file through the LFS protocol instead of regular Git object storage.

```
PS C:\Users\WangShiyu\test-lfs> git add .gitattributes
PS C:\Users\WangShiyu\test-lfs> git add large_file.bin
PS C:\Users\WangShiyu\test-lfs> git commit -m "Add 1GB large file using Git LFS"
Author identity unknown

*** Please tell me who you are.

Run

  git config --global user.email "you@example.com"
  git config --global user.name "Your Name"

to set your account's default identity.
Omit --global to set the identity only in this repository.

fatal: unable to auto-detect email address (got 'WangShiyu@LAPTOP-V2NB5L2H.(none)')
PS C:\Users\WangShiyu\test-lfs> git config user.email "202383910020@nuist.edu.cn"
PS C:\Users\WangShiyu\test-lfs> git config user.name "WangShiyu"
PS C:\Users\WangShiyu\test-lfs> git commit -m "Add 1GB large file using Git LFS"
[main (root-commit) b12ff75] Add 1GB large file using Git LFS
 2 files changed, 4 insertions(+)
 create mode 100644 .gitattributes
 create mode 100644 large_file.bin
PS C:\Users\WangShiyu\test-lfs> git push origin main
warning: use of unencrypted HTTP remote URLs is not recommended; see https://aka.ms/gcm/unsaferepotes for more informati
on.
Locking support detected on remote "origin". Consider enabling it with:
  $ git config lfs.http://localhost:3000/WangShiyu/test-lfs.git/info/lfs.locksverify true
Uploading LFS objects: 100% (1/1), 1.1 GB | 88 MB/s, done.
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 20 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (4/4), 438 bytes | 438.00 KiB/s, done.
Total 4 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote: . Processing 1 references
remote: Processed 1 references in total
To http://localhost:3000/WangShiyu/test-lfs.git
 * [new branch]      main -> main
PS C:\Users\WangShiyu\test-lfs> git lfs ls-files
49bc20df15 * large_file.bin
```

5. Verification of LFS Functionality

The Gitea web interface showed the file in the repository, confirming that the large file was successfully uploaded through the LFS system.

WangShiyu	b12ff75dd3	Add 1GB large file using Git LFS
.gitattributes		Add 1GB large file using Git LFS
large_file.bin		Add 1GB large file using Git LFS

Conclusion

Through this experiment, I successfully deployed a self-hosted Git service using Docker and verified its support for Git Large File Storage (LFS).

1. Successful Deployment

Gitea was successfully deployed using Docker Compose on Windows with WSL2. The containerization approach provided an isolated and reproducible environment, while Docker volumes ensured data persistence across container restarts.

2. Git LFS Functionality Verified

The experiment confirmed that Gitea supports Git LFS integration. By enabling the `GITEA__server__LFS_START_SERVER=true` environment variable, server-side LFS support was activated. The successful upload of a 1GB test file through the LFS protocol demonstrated proper functionality.

3. Key Learnings

The experiment demonstrated that Docker Compose simplifies application deployment and management, while Git LFS effectively solves the problem of version control for large files. Self-hosted solutions like Gitea offer a lightweight alternative to cloud-based platforms with full control over repositories.

4. Challenges Encountered

Minor issues such as command compatibility and Git user configuration were encountered and resolved during the experiment. These challenges provided practical learning experience in troubleshooting environment-specific problems.